

# Private Network Service Platform — Phase 1 Report

## Computer Networks Course Project · Phase 1: Build & Observe

**Team 1** · Prince Kumar Singh (Mac 1) · Atanu Adhikari (Mac 2) · Sambhav Kumar (Mac 3)

Build and evidence collected on 2 October 2026 (IST) on the college Wi-Fi Rishihood\_Learner.

Core principle of the project: *the application stays simple — the network is the project.*

## 1. Objective

Build a fully local private service on our own laptops, with no cloud, and prove every step of one request with tools:

1. A client types the private name `app.team1.test`.
2. The name is resolved by **our own DNS server** (Mac 1).
3. The client opens an **HTTPS** connection with no certificate warning to our **nginx edge** (Mac 2).
4. nginx load-balances the request to one of **two backends** (Mac 3, ports 3001 and 3002).
5. DNS, TCP, TLS and HTTP are all captured and explained with `dig`, `curl -v`, Wireshark and browser DevTools.

**Phase 1 gate (spec section 5):** a client resolves `app.teamX.test`, connects over HTTPS, and receives responses from both backends through the load balancer. **Status: met.** Mac 1 and Mac 3 each resolved `app.team1.test` to `10.7.16.45`, connected over HTTPS with SSL certificate verify ok, and received both `X-Backend: A` and `X-Backend: B`.

## 2. Team, roles and machine inventory (Task A)

Our team has 3 members. The spec's 4-role layout allows teams of 2–3 to combine roles (section 3), so **both backends run on Mac 3** as two separate processes on two ports.

| Machine | Owner   | Role                                                      | IPv4        | Mask / prefix       | Gateway  | Interface | Wi-Fi MAC                                                                                                                                     |
|---------|---------|-----------------------------------------------------------|-------------|---------------------|----------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Mac 1   | Prince  | Private DNS (dnsmasq) + test client                       | 10.7.28.177 | 255.255.224.0 (/19) | 10.7.0.1 | en0       | 10:9f:41:bd:94:ac                                                                                                                             |
| Mac 2   | Atanu   | Edge: nginx reverse proxy, TLS termination, load balancer | 10.7.16.45  | 255.255.224.0 (/19) | 10.7.0.1 | en0       | hardware 80:a9:97:48:ca:2f; on this network macOS uses the private address 12:32:e6:75:62:df (seen in ifconfig, the Wi-Fi menu and Wireshark) |
| Mac 3   | Sambhav | Backend A :3001 + Backend B                               | 10.7.7.111  | 255.255.224.0 (/19) | 10.7.0.1 | en0       | 10:9f:41:c2:3a:02                                                                                                                             |

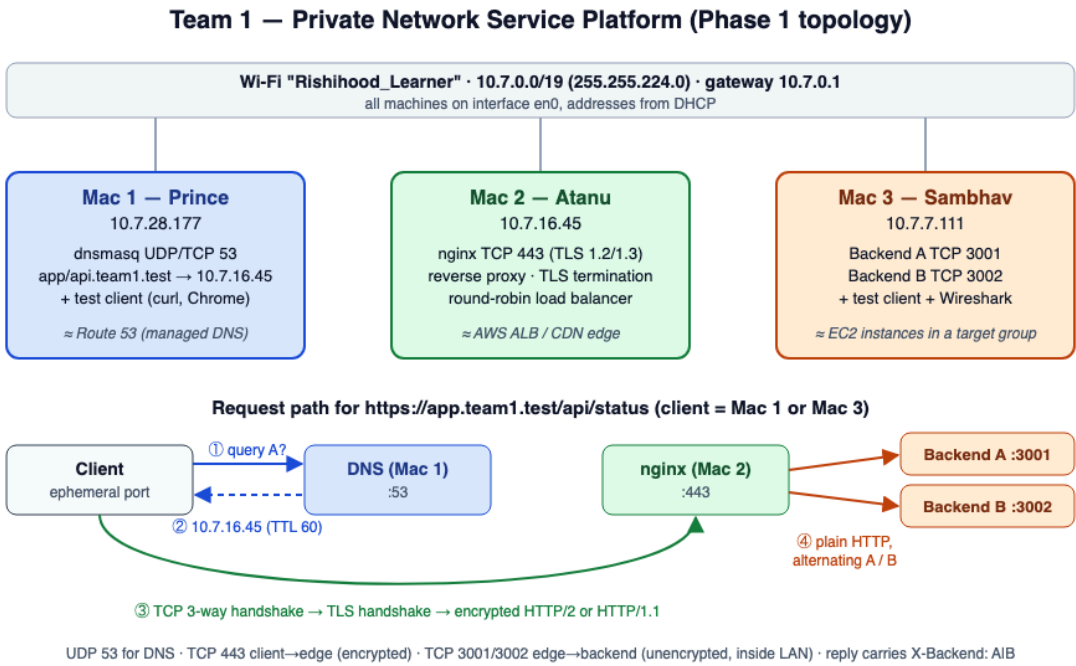
| Machine | Owner | Role                            | IPv4 | Mask / prefix | Gateway | Interface | Wi-Fi MAC |
|---------|-------|---------------------------------|------|---------------|---------|-----------|-----------|
|         |       | :3002 + test client + Wireshark |      |               |         |           |           |

Collected with `networksetup -getinfo Wi-Fi`, `ifconfig en0` and `route -n get default` on each Mac (evidence: [evidence/A-ping](#)).

**Reachability:** all six directed pings (1→2, 1→3, 2→1, 2→3, 3→1, 3→2) received 4 of 4 replies, **0.0 % packet loss**, TTL 64 (same subnet, no router hop). Round-trip times ranged from about 8.6 ms to 115 ms. That spread is normal on a shared college Wi-Fi, and it also explains the TCP retransmissions in our capture (section 6).

All IPs come from DHCP. At the start of every session we run `ipconfig getifaddr en0` on all three Macs. If Mac 2’s address changes, the `dnsmasq host-record` lines must change. If Mac 3’s address changes, the `nginx upstream` must change.

### 3. Topology and cloud mapping



The client only ever learns the **edge's** IP (10.7.16.45). The backend addresses live only in the nginx `upstream` block. That is why we can add, remove or replace backends without changing DNS or any client.

## 4. Request flow, ports and protocol layers

### 4.1 One request, step by step (`https://app.team1.test/api/status` from Mac 3)

| # | What happens                                                                                               | Protocol / layer    | Socket (from our capture)                                        |
|---|------------------------------------------------------------------------------------------------------------|---------------------|------------------------------------------------------------------|
| 1 | Client asks “A record for <code>app.team1.test</code> ?”                                                   | DNS over <b>UDP</b> | 10.7.7.111: <b>50215</b> → 10.7.28.177: <b>53</b> (pkt 239)      |
| 2 | dnsmasq answers 10.7.16.45, TTL 60                                                                         | DNS / UDP           | 10.7.28.177:53 → 10.7.7.111:50215 (pkt 240, ~10 ms later)        |
| 3 | ARP: “who has 10.7.16.45?” → 12:32:e6:75:62:df                                                             | ARP, link layer     | broadcast (pkts 241–242)                                         |
| 4 | TCP three-way handshake SYN → SYN-ACK → ACK                                                                | TCP                 | 10.7.7.111: <b>59200</b> → 10.7.16.45: <b>443</b> (pkts 243–247) |
| 5 | TLS handshake: ClientHello (SNI <code>app.team1.test</code> , ALPN <code>h2/http1.1</code> ) ... Finished  | TLS over TCP        | same connection (pkts 249–256)                                   |
| 6 | Encrypted HTTP/2 request and response                                                                      | HTTP inside TLS     | Application Data (pkts 258–278)                                  |
| 7 | nginx opens its own TCP connection to the chosen backend and forwards the request in <b>plain HTTP/1.1</b> | TCP + HTTP          | 10.7.16.45: <b>55239</b> → 10.7.7.111: <b>3001</b> (pkt 265)     |
| 8 | Backend answers (HTTP/1.0, <code>X-Backend: A</code> ); nginx re-encrypts toward the client                | HTTP → TLS          | —                                                                |
| 9 | Connection closed: Encrypted Alert, then FIN/ACK in both directions                                        | TLS / TCP           | pkts 281–286                                                     |

### 4.2 Ports table

| Service       | Machine | Port / transport                                           | Who connects                    |
|---------------|---------|------------------------------------------------------------|---------------------------------|
| DNS (dnsmasq) | Mac 1   | 53 UDP (and TCP)                                           | every client resolver           |
| HTTPS (nginx) | Mac 2   | 443 TCP                                                    | clients                         |
| Backend A     | Mac 3   | 3001 TCP                                                   | nginx only, in normal use       |
| Backend B     | Mac 3   | 3002 TCP                                                   | nginx only, in normal use       |
| Client side   | any     | ephemeral (e.g. 50215 UDP, 59200 TCP; nginx side 55239...) | chosen by the OS per connection |

A **socket pair** (src IP, src port, dst IP, dst port, protocol) identifies each connection. The client→edge connection and the edge→backend connection are **two separate TCP connections**. That is what “reverse proxy” means.

### 4.3 OSI vs TCP/IP mapping

| OSI layer                  | TCP/IP layer                                | In our project                                                                    |
|----------------------------|---------------------------------------------|-----------------------------------------------------------------------------------|
| 7 Application              | Application                                 | DNS query/answer, HTTP/1.1 and HTTP/2 (REST JSON, X-Backend, Cache-Control, ETag) |
| 6 Presentation / 5 Session | Application (TLS sits between HTTP and TCP) | TLS 1.2 / 1.3: certificate, key exchange, encryption                              |
| 4 Transport                | Transport                                   | UDP 53 for DNS; TCP 443 and TCP 3001/3002 (handshake, seq/ack, retransmission)    |
| 3 Network                  | Internet                                    | IPv4 10.7.0.0/19; TTL 64 in pings                                                 |
| 2 Data link / 1 Physical   | Link                                        | Wi-Fi 802.11ax, Ethernet II framing, MAC addresses, ARP                           |

## 5. Implementation of Tasks B–F

### Task B — Private DNS server (Mac 1)

dnsmasq 2.93, config at /opt/homebrew/etc/dnsmasq.conf ([copy](#), [explanation](#)):

```
interface=en0
listen-address=127.0.0.1
domain-needed
bogus-priv
local=/team1.test/
host-record=app.team1.test,10.7.16.45
host-record=api.team1.test,10.7.16.45
local-ttl=60
no-resolv
server=8.8.8.8
server=1.1.1.1
log-queries
log-facility=/tmp/dnsmasq.log
```

- `host-record` gives an exact A record for each name. Both names point to **Mac 2 (the edge)**, never to a backend.
- `local=/team1.test/` makes dnsmasq authoritative for our zone. Other names are forwarded to 8.8.8.8 / 1.1.1.1, so clients keep internet access.
- Client resolvers: Mac 2 and Mac 3 set to 10.7.28.177 (System Settings → Wi-Fi → Details → DNS, Search Domains empty). Mac 1 set to 127.0.0.1.

**Results** ([evidence/B-dns](#)): - Mac 2, 12:41:59 IST: `dig app.team1.test → app.team1.test. 60 IN A 10.7.16.45, SERVER: 10.7.28.177#53, flags qr aa rd ra` (authoritative answer). - Mac 3, 12:41:59 IST: same answer from 10.7.28.177. - Mac 3, 12:46:50 IST: `dig api.team1.test → 60 IN A 10.7.16.45. - dig @8.8.8.8 app.team1.test → NXDOMAIN`: the name exists only inside our network.

**DNS vs connection:** DNS only turns a name into an IP. It moves no application data. After the answer, the client opens a completely separate TCP connection to that IP on port 443. In the capture the DNS exchange (UDP) ends at packet 240 and the TCP SYN starts at packet 243.

### Task C — Two backend services (Mac 3)

`backend/backend.py` is one Python standard-library script started twice: `python3 backend.py A 3001` and `python3 backend.py B 3002`.

| Endpoint        | Response                                                                                                                 |
|-----------------|--------------------------------------------------------------------------------------------------------------------------|
| GET /           | <code>{"service":"team1 backend", "backend":"A", "message":"Backend A is running"}</code>                                |
| GET /api/status | <code>{"backend":"A", "status":"ok", "time":"..."}</code> , Cache-Control: no-store                                      |
| GET /api/info   | Same JSON on both backends, Cache-Control: public, max-age=60, ETag: "6c623c0954f8001c", 304 on a matching If-None-Match |
| every response  | X-Backend: A or X-Backend: B                                                                                             |

Both bind to `0.0.0.0`, so `lsof` shows `*:3001` and `*:3002` (not `127.0.0.1` only). Mac 2 reached both over the LAN (`curl -i http://10.7.7.111:3001/api/status` → X-Backend: A, `:3002` → B). The backend terminal logs show requests from `127.0.0.1` (local test) and from `10.7.16.45` (nginx) ([evidence/C-backends](#)).

### Task D — Edge reverse proxy and load balancer (Mac 2)

nginx 1.31.6. The server block is in `/opt/homebrew/etc/nginx/servers/team1.conf` ([copy](#)):

```
log_format team1 '$remote_addr -> $upstream_addr [$time_local] "$request" $status';

upstream team1_backends {
    server 10.7.7.111:3001 max_fails=1 fail_timeout=10s;    # Backend A
    server 10.7.7.111:3002 max_fails=1 fail_timeout=10s;    # Backend B
}

server {
    listen 443 ssl;
    http2 on;
    server_name app.team1.test api.team1.test;
    ssl_certificate      /opt/homebrew/etc/nginx/certs/app.team1.test.crt;
    ssl_certificate_key  /opt/homebrew/etc/nginx/certs/app.team1.test.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
    access_log /opt/homebrew/var/log/nginx/team1_access.log team1;

    location / {
        proxy_pass http://team1_backends;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_connect_timeout 2s;
        proxy_next_upstream error timeout http_502 http_503;
    }
}
```

- **Strategy:** round-robin (nginx's default). Each new request goes to the next server in the list.
- **Passive health check:** after 1 failed attempt a server is marked down for 10 s. `proxy_next_upstream` retries the failed request on the other server, so the client still gets a 200.
- **Results:** a loop of 6 requests from Mac 1, Mac 2 and Mac 3 printed `A, B, A, B, ...`. The nginx access log ([evidence/D-loadbalancing](#)) holds 122 requests from all three clients: **62 served by :3001 and 61 by :3002**, for example:

```
10.7.7.111 -> 10.7.7.111:3002 [02/Oct/2026:13:44:54 +0530] "GET /api/status HTTP/2.0" 200
10.7.7.111 -> 10.7.7.111:3001 [02/Oct/2026:13:44:54 +0530] "GET /api/status HTTP/2.0" 200
10.7.7.111 -> 10.7.7.111:3002 [02/Oct/2026:13:44:55 +0530] "GET /api/status HTTP/2.0" 200
```

**Why the client never needs backend IPs:** DNS gives out only the edge address. The client's TCP connection ends at nginx, and nginx alone decides which backend to use. The client can see which backend answered only because of the `X-Backend` header we added on purpose.

## Task E — HTTPS / TLS

We used our **own team CA** instead of a bare self-signed certificate, so every client can trust it properly without `-k` ([setup notes](#)).

| Item               | Value                                                                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| CA                 | CN=Team1 Local CA, O=Team1 CN Project, C=IN; RSA 2048, CA:TRUE, keyCertSign                                                               |
| Server certificate | CN=app.team1.test, issued by Team1 Local CA                                                                                               |
| SAN                | DNS:app.team1.test, DNS:api.team1.test                                                                                                    |
| EKU / validity     | serverAuth; 2 Oct 2026 07:40:15 GMT → 2 Oct 2027                                                                                          |
| Check              | <code>openssl verify -CAfile team1-ca.crt app.team1.test.crt</code> → <b>OK</b>                                                           |
| Trust              | <code>team1-ca.crt</code> added as <code>trustRoot</code> to the System keychain on <b>all three Macs</b> ; private keys never left Mac 2 |
| nginx              | TLS terminates at nginx on 443, TLS 1.2 + 1.3, HTTP/2 enabled                                                                             |

**Results** ([evidence/E-tls](#)), from `/usr/bin/curl -v` on Mac 3:

|                        | TLS 1.2 ( <code>--tls-max 1.2</code> )                                                                                                         | TLS 1.3 (default)                                                                                         |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Handshake seen by curl | Client hello → Server hello → <b>Certificate</b> → Server key exchange → Server finished → Client key exchange → Change cipher spec → Finished | Client hello → Server hello → Encrypted Extensions ("Unknown (8)") → Certificate → CERT verify → Finished |
| Negotiated             | TLSv1.2 / ECDHE-RSA-CHACHA20-POLY1305                                                                                                          | TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256                                                                   |
| ALPN                   | server accepted <b>h2</b>                                                                                                                      | server accepted <b>h2</b>                                                                                 |
| Verification           | <code>subjectAltName: host "app.team1.test" matched · SSL certificate verify ok</code>                                                         | same                                                                                                      |
| Response               | HTTP/2 200, x-backend: A                                                                                                                       | HTTP/2 200, x-backend: B                                                                                  |

Also shown: Mac 1 `curl -v https://app.team1.test` (TLS 1.3, verify ok, HTTP/2 200), Chrome opening `https://app.team1.test/api/status` by name with no warning, `curl -sI --http1.1` → HTTP/1.1 200 OK and `--http2` → HTTP/2 200. **HTTP/3** (QUIC over UDP) is explanation-only, as the spec allows.

**Handshake in words:** ClientHello (supported versions and ciphers, SNI, ALPN, key share) → ServerHello (chosen cipher; in our capture 0xcca8 for TLS 1.2 and 0x1303 for TLS 1.3) → Certificate (server proves its identity with a chain to a CA the client trusts) → Key Exchange (ECDHE, so both sides derive the same session keys) → ChangeCipherSpec / Finished (everything after this is encrypted). In TLS 1.3 the certificate itself is already encrypted, which is why we also captured a TLS 1.2 session to see the Certificate in clear.

## Task F — HTTP caching

`/api/info` returns `Cache-Control: public, max-age=60` and an `ETag` built from a hash of the body, so Backend A and Backend B produce the **same** `ETag` ([evidence/F-caching](#)).

```
$ curl -I https://app.team1.test/api/info           (Mac 1, 11:41:03 GMT)
HTTP/2 200
server: nginx/1.31.6
content-type: application/json
content-length: 117
x-backend: B
cache-control: public, max-age=60
etag: "6c623c0954f8001c"

$ curl -i -H "If-None-Match: $ETAG" https://app.team1.test/api/info
HTTP/2 304                                     (x-backend: A at 11:42:31 and 11:45:45, x-backend: B at 11:45:48)
```

- The **304 Not Modified** came from both A and B. Round-robin does not break revalidation because the `ETag` is content-based.
- **Chrome DevTools:** a repeated `fetch` was served 200 (disk cache) in **1 ms** (no network at all). Reloading the page sent a conditional request that got **304** (0.1 kB).
- **nginx log** shows the same pattern: `"GET /api/info HTTP/2.0" 304` lines, plus 200 when the cache was empty or expired.
- `/api/status` uses `Cache-Control: no-store`, so live status is never cached.

| Case                | What happens on the network                                                                                    | Our evidence                         |
|---------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------|
| Fresh cache hit     | Response younger than max-age → served locally, <b>no request sent</b>                                         | DevTools (disk cache), 1 ms          |
| Conditional request | Copy expired or reload → client sends <code>If-None-Match</code> , server replies <b>304</b> with headers only | <code>curl 304</code> , DevTools 304 |
| Full new request    | No cached copy → full <b>200</b> with body                                                                     | <code>curl -I 200</code> , log 200   |

## 6. Task G — Packet capture analysis

Captured on **Mac 3, Wi-Fi en0**, no capture filter. Saved as `phase1_full_flow.pcapng` (683 packets, 483 s). Because Mac 3 is both the client and the backend host, the same file also holds all six `nginx→backend` connections, in plain text. A machine-readable summary made with `tshark` is in `phase1_full_flow_tshark_summary.txt`. Traffic: DNS cache flushed, then `/usr/bin/curl -v --tls-max 1.2 .../api/status`, then TLS 1.3, then a 4-request loop ([evidence/G-capture](#)).

| Layer / event     | Display filter                                                       | What we found                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DNS               | <code>dns.qry.name contains "team1.test"</code>                      | Pkt 239 (t = 101.203 s) standard query 0x647b A app.team1.test, UDP <b>50215</b> → <b>53</b> . Pkt 240 (t = 101.214 s) response <b>A 10.7.16.45, TTL 60</b> , authoritative flag set. Second lookup pkts 392/393 (0x5868). Conversations: all 210 UDP conversations of Mac 3 go to 10.7.28.177:53. The team1.test ones use source ports 50215 and 51388                              |
| ARP               | —                                                                    | Pkt 241 “Who has 10.7.16.45? Tell 10.7.7.111”, pkt 242 “10.7.16.45 is at 12:32:e6:75:62:df”: IP → MAC before the first frame to the edge                                                                                                                                                                                                                                             |
| TCP handshake     | <code>tcp.flags.syn==1, tcp.stream eq 0</code>                       | Pkt 243 <b>SYN</b> 59200 → 443, Seq=0, Win=65535, MSS=1460, WS=64, SACK_PERM · pkt 244 SYN retransmission · pkt 245 <b>SYN-ACK</b> Seq=0 Ack=1 · pkt 246 duplicate SYN-ACK · pkt 247 <b>ACK</b> Seq=1 Ack=1 · pkt 248 Dup ACK. Only then does TLS start (pkt 249)                                                                                                                    |
| TLS 1.2           | <code>tcp.stream eq 0 &amp;&amp; tls</code>                          | 249 Client Hello (SNI=app.team1.test, 46 cipher suites, ALPN h2/http/1.1) · 252 <b>Server Hello, Certificate, Server Key Exchange, Server Hello Done</b> (tls.handshake.type == 11 → only pkt 252) · 254 Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message · 256 Change Cipher Spec, Encrypted Handshake Message · 258–278 Application Data · 281 Encrypted Alert |
| TLS 1.3           | <code>tcp.stream eq 2 &amp;&amp; tls</code>                          | Pkt 399 Client Hello from port <b>59239</b> (TLS 1.3 suites incl. TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256) · 402 Server Hello, Change Cipher Spec, Application Data. The certificate is no longer visible because TLS 1.3 encrypts it                                                                                                           |
| Encrypted payload | <code>tls.record.content_type == 23</code>                           | Pkts 258, 259, 260, 262, 266, 278 “Application Data”. The hex pane shows unreadable bytes, so the HTTP headers that <code>curl -v</code> prints are not visible on the wire                                                                                                                                                                                                          |
| Edge → backend    | <code>tcp.port == 3001 or tcp.port == 3002, Follow TCP Stream</code> | Pkt 265 nginx SYN 10.7.16.45:55239 → 10.7.7.111:3001; request pkt 269, response pkt 274. Follow TCP Stream shows <b>plain text</b> : GET /api/status HTTP/1.1, Host: app.team1.test, X-Real-IP: 10.7.7.111, X-                                                                                                                                                                       |

| Layer / event  | Display filter                        | What we found                                                                                                                                                                                                                                 |
|----------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                |                                       | Forwarded-Proto: https → HTTP/1.0 200 OK, Server: TeamBackend/1.0 Python/3.12.4, X-Backend: A. The six backend responses (pkts 274, 425, 505, 545, 585, 626) carry <b>X-Backend A, B, A, B, A, B</b> . This proves TLS terminates at the edge |
| Load balancing | Statistics → Conversations → TCP (12) | Six client→443 connections (source ports 59200, 59239, 59247–59250) and six nginx→backend connections (55239, 55245–55249) <b>alternating 3001 / 3002</b>                                                                                     |
| Teardown       | tcp.stream eq 0                       | Pkts 282–286 FIN/ACK in both directions (client Seq=524, Ack=1713)                                                                                                                                                                            |

**Sequence / acknowledgement numbers (reliability):** the numbers are relative. SYN uses Seq 0. Each side's ACK number is “the next byte I expect”. After the ClientHello (231 bytes of TCP payload, Seq=1), the server acknowledges **Ack=232**. After the server's 1381-byte flight the client acknowledges Ack=1382. By the end the client has sent 523 bytes (Seq=524) and received 1712 (Ack=1713). Pkts 244 and 246 are **retransmissions**: on this busy Wi-Fi the first SYN/SYN-ACK was not acknowledged in time, so TCP re-sent it. The connection still completed, which is TCP's reliable delivery at work and not an error. The `Win=` field (e.g. 131712 after window scaling) is TCP **flow control**: how many bytes the receiver can accept.

## 7. Phase 1 failure demonstrations (spec 6.3)

The table gives the procedure we follow for each scenario and the result we expect, based on how our system is built. The status column says honestly which ones are already recorded.

| # | Scenario                            | How we break it                                                                                        | Expected observation                                                                                                                                                                                                                    | Restore                             | Status                                                                                                                                     |
|---|-------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Wrong DNS server on a client</b> | Mac 3: <code>sudo networksetup -setdnsservers Wi-Fi 8.8.8.8</code> + flush                             | <code>dig app.team1.test</code> → <b>NXDOMAIN</b> from 8.8.8.8. <code>curl</code> → “Could not resolve host”. <b>But ping 10.7.16.45 works.</b> DNS and IP connectivity are independent layers                                          | set back to 10.7.28.177 + flush     | Seen by accident during setup (Mac 2/3 had no DNS set → NXDOMAIN via 8.8.8.8); <b>deliberate recording pending</b>                         |
| 2 | <b>DNS record → wrong IP</b>        | Mac 1: change <code>host-record=app.team1.test,...</code> to 10.7.7.111, restart dnsmasq, flush client | <code>dig</code> succeeds (NOERROR, answer 10.7.7.111), but <code>curl https://app.team1.test</code> → “Failed to connect ... port 443: Connection refused”, because nothing listens on 443 there. DNS is a directory, not a connection | put back 10.7.16.45, restart, flush | Wrong answer 10.7.7.111 captured by accident at 12:12 IST on Mac 2 and Mac 3 ( <a href="#">evidence</a> ); <b>curl consequence pending</b> |

| # | Scenario                                                  | How we break it                             | Expected observation                                                                                                                                                                                                                                                                                                       | Restore                                                    | Status                                      |
|---|-----------------------------------------------------------|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|---------------------------------------------|
| 3 | <b>One backend stopped</b><br>(our form choice: Option A) | Mac 3: Ctrl+C Backend A                     | Loop keeps returning <b>200</b> , only <b>X-Backend: B</b> . nginx error.log: connect() failed (61: Connection refused) ... upstream: "http://10.7.7.111:3001/..."; max_fails=1 marks A down for 10 s and proxy_next_upstream retries on B. From Mac 2, curl http://10.7.7.111:3001 is refused while ping 10.7.7.111 works | python3 backend.py A 3001; after ~10 s A/B alternate again | <b>Pending</b><br>(record for the D3 video) |
| 4 | <b>Both backends stopped</b>                              | Mac 3: Ctrl+C A and B                       | curl -v: DNS ✓, TCP ✓, TLS ✓ (verify ok), then <b>HTTP/2 502</b> generated by nginx itself: no X-Backend header, server: nginx. This is where the edge ends and the backend begins                                                                                                                                         | restart both                                               | <b>Pending</b>                              |
| 5 | <b>Wrong destination port</b>                             | Client: curl -v https://app.team1.test:9443 | "Failed to connect to app.team1.test port 9443 ... Connection refused" (a TCP RST). nc -vz app.team1.test 443 succeeds, 9443 is refused, ping works. IP finds the host, the port finds the service                                                                                                                         | —                                                          | <b>Pending</b>                              |

**Related unplanned observation (already in our Mac 2 logs, 2 Oct 23:52 IST):** when Mac 3 was not reachable at all, nginx logged upstream timed out (60: Operation timed out) while connecting to upstream for **both** 3002 and 3001, then returned **504 Gateway Timeout**. Compare with scenario 4: a backend host that is *up but has no process on the port* answers with a TCP RST, so nginx gives **502 Bad Gateway** at once. A host that is *gone* sends nothing, so nginx waits for proxy\_connect\_timeout (2 s per server) and gives **504**.

## 8. Problems we hit and how we fixed them

| # | Problem                                                    | Root cause                                                                  | Fix / lesson                                                                                                                       |
|---|------------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Could not clear the ~700-line default dnsmasq.conf in nano | Wrong tool for a full rewrite                                               | Overwrote it with cat > file <<'EOF'                                                                                               |
| 2 | dig still returned the old IP after an edit                | brew services start does nothing when the service is already running        | Always sudo brew services restart dnsmasq                                                                                          |
| 3 | DNS answered 10.7.7.111 (Mac 3) instead of the edge        | Wrong IP typed into host-record, and it was accepted without cross-checking | Confirmed Mac 2 = 10.7.16.45 with ipconfig getifaddr en0, fixed, re-verified from every Mac. Kept as failure evidence (scenario 2) |

| #  | Problem                                                           | Root cause                                                                                                    | Fix / lesson                                                                                               |
|----|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 4  | Mac 2/3 dig → NXDOMAIN with SERVER: 8.8.8.8                       | Their system DNS had never been set                                                                           | networksetup – setdnsservers Wi-Fi 10.7.28.177 + cache flush                                               |
| 5  | 8.8.4.4 appeared under <b>Search Domains</b> on one Mac           | IP typed in the wrong box                                                                                     | Removed it. Search Domains stay empty                                                                      |
| 6  | Error reading file ~/Downloads/team1-ca.crt on Mac 2              | Ran the Mac 1/3 command; on Mac 2 the CA is in ~/cn-project/certs                                             | Used the local path                                                                                        |
| 7  | Mac 1 curl → “Could not resolve host” while dig @127.0.0.1 worked | dig @server bypasses the system resolver, and Mac 1’s own system DNS was empty                                | Set Mac 1 DNS to 127.0.0.1. Lesson: test with plain dig / dscacheutil, which is what curl and browsers use |
| 8  | Mac 3 load-balancing loop printed nothing                         | curl was Anaconda’s (/opt/anaconda3/bin/curl), which ignores the macOS Keychain; –s hid the certificate error | Use /usr/bin/curl on Mac 3                                                                                 |
| 9  | Chrome on Mac 1: ERR_ADDRESS_UNREACHABLE                          | macOS Local Network privacy permission for Chrome                                                             | Privacy & Security → Local Network → Chrome ON, then restart Chrome                                        |
| 10 | Wireshark filter bar turned red                                   | Capture-filter syntax (host ...) typed as a display filter                                                    | Display filter ip.addr == ...                                                                              |
| 11 | Conversations showed 1 row and no ports                           | “Limit to display filter” ticked; window scrolled right                                                       | Unticked, scrolled left                                                                                    |
| 12 | dns filter showed only Google/Apple lookups                       | Mac 3 sends <b>all</b> its DNS to Mac 1                                                                       | dns.qry.name contains "team1.test"                                                                         |
| 13 | Red retransmission rows in the handshake                          | Real Wi-Fi delay                                                                                              | Not a bug. Used as TCP reliability evidence                                                                |

## 9. Key learnings

- **Layers really are independent.** DNS working says nothing about TCP. TCP working says nothing about TLS. TLS working says nothing about the backend. Our debugging always went name resolution → TCP connect → TLS → HTTP.
- **DNS is a directory, not a connection.** A wrong record still “works” at the DNS layer. It just sends you to the wrong place.
- **The edge hides the backends.** One name, one IP and one certificate for the client. Any number of backends, health checks and plain HTTP stay behind it.
- **TLS termination** means encryption ends at nginx. Wireshark proved it: ciphertext on 443, readable HTTP on 3001/3002. Trust comes from the CA being in each client’s keychain, not from the certificate alone.
- **Caching** saves round trips (fresh hit, no request at all) or bandwidth (304, headers only). A content-based ETag keeps revalidation working behind a load balancer.
- **Remaining single point of failure:** nginx on Mac 2, and also dnsmasq on Mac 1. Removing them is Phase 2 (backup DNS, standby edge with DNS cutover).

## 10. Phase 1 checklist

| Task                                                                       | Status                             |
|----------------------------------------------------------------------------|------------------------------------|
| A – LAN (IPs, masks, gateway, interface, MAC, pings, topology diagram)     | ✓                                  |
| B – Private DNS, 2 client Macs resolving through Mac 1, app + api records  | ✓                                  |
| C – Two backends on LAN interface, /, /api/status, X-Backend               | ✓                                  |
| D – nginx round-robin, A/B alternation proven                              | ✓                                  |
| E – HTTPS with team CA, trusted on all clients, no -k, HTTP/1.1 + HTTP/2   | ✓                                  |
| F – Cache-Control, ETag, 304, disk-cache hit                               | ✓                                  |
| G – DNS / TCP / TLS / encrypted data / ports / LB captured                 | ✓                                  |
| 6.3 – Five failure demonstrations                                          | <input type="checkbox"/> to record |
| Faculty confirmation of team number ( team1 ) and the 3-machine role split | <input type="checkbox"/>           |

## Appendix – repository map

| Deliverable (spec section 9)              | Where                                                                 |
|-------------------------------------------|-----------------------------------------------------------------------|
| Architecture document                     | this report + <a href="#">diagrams/topology.svg</a>                   |
| Configuration bundle                      | <a href="#">config/</a> : dnsmasq, nginx, TLS notes, client DNS setup |
| Backend source code + launch instructions | <a href="#">backend/</a> , helper scripts in <a href="#">scripts/</a> |
| Evidence folder                           | <a href="#">evidence/</a> , organised A–G + failures                  |